

SOC DSA Week -1 Submission:

1.Set Matrix Zeroes

</> Code

C++ ▾ 🔒 Auto

```

1  class Solution {
2  public:
3      void setZeroes(vector<vector<int>>& matrix) {
4          int m = matrix.size();
5          int n = matrix[0].size();
6          bool firstRowZero = false;
7          bool firstColZero = false;
8
9          for (int j = 0; j < n; j++) {
10             if (matrix[0][j] == 0) firstRowZero = true;
11         }
12
13         for (int i = 0; i < m; i++) {
14             if (matrix[i][0] == 0) firstColZero = true;
15         }
16         for (int i = 1; i < m; i++) {
17             for (int j = 1; j < n; j++) {
18                 if (matrix[i][j] == 0) {
19                     matrix[i][0] = 0;
20                     matrix[0][j] = 0;
21                 }
22             }
23         }
24         for (int i = 1; i < m; i++) {
25             for (int j = 1; j < n; j++) {
26                 if (matrix[i][0] == 0 || matrix[0][j] == 0) {
27                     matrix[i][j] = 0;
28                 }
29             }
30         }
31         if (firstRowZero) {
32             for (int j = 0; j < n; j++) {
33                 matrix[0][j] = 0;
34             }
35         }
36
37         if (firstColZero) {
38             for (int i = 0; i < m; i++) {
39                 matrix[i][0] = 0;
40             }
41         }
42     }

```

Saved

```
    }  
    if (firstRowZero) {  
        for (int j = 0; j < n; j++) {  
            matrix[0][j] = 0;  
        }  
    }  
  
    if (firstColZero) {  
        for (int i = 0; i < m; i++) {  
            matrix[i][0] = 0;  
        }  
    }  
}  
};
```

2.Pascal Triangle

```
class Solution {
public:
    int pascalTriangleI(int r, int c) {
        int n = r - 1;
        int k = c - 1;

        long long res = 1;
        if (k > n - k) {
            k = n - k;
        }
        for (int i = 0; i < k; i++) {
            res = res * (n - i);
            res = res / (i + 1);
        }

        return (int)res;
    }
};
```

Description Editorial Submissions Discussion

Next Permutation

Subscribe to TUF+

Hints

Company

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

For example, for arr = [1,2,3], the following are all the permutations of arr: [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1].

The next permutation of an array of integers is the next lexicographically greater permutation of its integers.

More formally, if all the permutations of the array are sorted in lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted order.

If such arrangement is not possible (i.e., the array is the last permutation), then rearrange it to the lowest possible order (i.e., sorted in ascending order).

You must rearrange the numbers in-place and use only constant extra memory.

Example 1

Input: nums = [1,2,3]
Output: [1,3,2]
Explanation:
The next permutation of [1,2,3] is [1,3,2].

Example 2

Input: nums = [3,2,1]
Output: [1,2,3]
Explanation:
[3,2,1] is the last permutation. So we return the first: [1,2,3].

Now your turn!

Input: nums = [1,1,5]
Output: [1,5,1]

Test Cases

C++

```
1 class Solution {
2 public:
3     void nextPermutation(vector<int>& nums) {
4         // Your code goes here
5         int n = nums.size();
6         int ind = -1;
7
8         for (int i = n - 2; i >= 0; i--) {
9             if (nums[i] < nums[i + 1]) {
10                 ind = i;
11                 break;
12             }
13         }
14
15         if (ind == -1) {
16             reverse(nums.begin(), nums.end());
17             return;
18         }
19
20         for (int i = n - 1; i > ind; i--) {
21             if (nums[i] > nums[ind]) {
22                 swap(nums[i], nums[ind]);
23                 break;
24             }
25         }
26
27         reverse(nums.begin() + ind + 1, nums.end());
28     }
29 }
```

DescriptionEditorialSubmissionsDiscussion

Kadane's Algorithm

Subscribe to TUF+HintsCompany

Given an integer array `nums`, find the subarray with the largest sum and return the sum of the elements present in that subarray.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input: `nums = [2, 3, 5, -2, 7, -4]`
Output: 15
Explanation:
The subarray from index 0 to index 4 has the largest sum = 15

Example 2

Input: `nums = [-2, -3, -7, -2, -10, -4]`
Output: -2
Explanation:
The element on index 0 or index 3 make up the largest sum when taken as a subarray

Now your turn!

Input: `nums = [-1, 2, 3, -1, 2, -6, 5]`
Output: Pick your answer

57

86

Constraints

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`

C++

Tab-1 +

```
1 class Solution {
2 public:
3     int maxSubArray(vector<int>& nums) {
4         int maxi = INT_MIN;
5         int sum = 0;
6
7         for (int i = 0; i < nums.size(); i++) {
8             sum += nums[i];
9
10            if (sum > maxi) {
11                maxi = sum;
12            }
13
14            if (sum < 0) {
15                sum = 0;
16            }
17        }
18        return maxi;
19    }
20 };
```

Test Cases

Case 1Case 2+

Accepted

nums

DescriptionEditorialSubmissionsDiscussion

Sort an array of 0's 1's and 2's

Subscribe to TUF+HintsCompany

Given an array `nums` consisting of only 0, 1, or 2. Sort the array in non-decreasing order.

The sorting must be done in-place, without making a copy of the original array.

Example 1

Input: `nums = [1, 0, 2, 1, 0]`
Output: `[0, 0, 1, 1, 2]`
Explanation:
The `nums` array in sorted order has 2 zeroes, 2 ones and 1 two

Example 2

Input: `nums = [0, 0, 1, 1, 1]`
Output: `[0, 0, 1, 1, 1]`
Explanation:
The `nums` array in sorted order has 2 zeroes, 3 ones and zero twos

Now your turn!

Input: `nums = [1, 1, 2, 2, 1]`
Output: Pick your answer

[1, 1, 2, 2]

[1, 1, 2, 2, 2]

[2, 2, 1, 1, 1]

[1, 1, 2, 2, 1]

Constraints

- `1 <= nums.length <= 105`
- `nums` consists of 0, 1 and 2 only.

C++

Tab-1 +

```
1 class Solution {
2 public:
3     void sortZeroOneTwo(vector<int>& nums) {
4         int low = 0;
5         int mid = 0;
6         int high = nums.size() - 1;
7
8         while (mid <= high) {
9             if (nums[mid] == 0) {
10                 swap(nums[low], nums[mid]);
11                 low++;
12                 mid++;
13             } else if (nums[mid] == 1) {
14                 mid++;
15             } else {
16                 swap(nums[mid], nums[high]);
17                 high--;
18             }
19         }
20     }
21 };
```

Test Cases

Case 1Case 2+

Accepted

nums

[1, 0, 2, 1, 0]

YOUR OUTPUT

EXPECTED OUTPUT

Best time to buy and sell stock

Subscribe to TUF+

Hints

Company

Given an array `arr` of `n` integers, where `arr[i]` represents price of the stock on the `i`th day. Determine the **maximum profit** achievable by buying and selling the stock at most once.

The stock should be purchased before selling it, and both actions cannot occur on the same day.

Example 1

Input: `arr = [10, 7, 5, 8, 11, 9]`

Output: 6

Explanation: Buy on day 3 (price = 5) and sell on day 5 (price = 11), profit = 11 - 5 = 6.

Example 2

Input: `arr = [5, 4, 3, 2, 1]`

Output: 0

Explanation: In this case, no transactions are made. Therefore, the maximum profit remains 0.

Now your turn!

Input: `arr = [3, 8, 1, 4, 6, 2]`

Output: **Pick your answer**

☐ 7

☐ 5

☐ 3

☐ 6

Constraints

$1 \leq \text{arr.length} \leq 10^5$

$0 \leq \text{arr}[i] \leq 10^4$

Tab-1

+

```
1 class Solution{
2 public:
3     int stockBuySell(vector<int> arr, int n){
4         int maxProfit = 0;
5         int mini = arr[0];
6
7         for (int i = 1; i < n; i++) {
8             int cost = arr[i] - mini;
9
10            if (cost > maxProfit) {
11                maxProfit = cost;
12            }
13
14            if (arr[i] < mini) {
15                mini = arr[i];
16            }
17        }
18
19        return maxProfit;
20    }
21 };
22
23
```

Test Cases

✓ Case 1

✓ Case 2

+

Reset

Accepted

ARR

[10, 7, 5, 8, 11, 9]

Rotate matrix by 90 degrees

Subscribe to TUF+

Hints

Company

Given an `N * N` 2D integer matrix, rotate the matrix by 90 degrees clockwise.

The rotation must be done in place, meaning the input 2D matrix must be modified directly.

Example 1

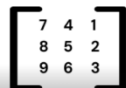
Input: `matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

Input:



Output: `matrix = [[7, 4, 1], [8, 5, 2], [9, 6, 3]]`

Output:



Tab-1

+

```
1 class Solution {
2 public:
3     void rotateMatrix(vector<vector<int>>& matrix) {
4         int n = 0;
5         for (int i = 0; i < matrix.size(); i++) {
6             n++;
7         }
8         for (int i = 0; i < n; i++) {
9             for (int j = 0; j < i; j++) {
10                 int temp = matrix[i][j];
11                 matrix[i][j] = matrix[j][i];
12                 matrix[j][i] = temp;
13             }
14         }
15         for (int i = 0; i < n; i++) {
16             int start = 0;
17             int end = n - 1;
18             while (start < end) {
19                 int temp = matrix[i][start];
20                 matrix[i][start] = matrix[i][end];
21                 matrix[i][end] = temp;
22                 start++;
23                 end--;
24             }
25         }
26     }
27 };
28
```

Test Cases

✓ Case 1

✓ Case 2

+

Reset

Accepted

NUMS

3.Merge Overlapping Subintervals

```
C++
Tab-1 +
1 class Solution {
2     public:
3         vector<vector<int>> mergeOverlap(vector<vector<int>>& arr) {
4             // Your code goes here
5             int n = 0;
6             for (int i = 0; i < arr.size(); i++) {
7                 n++;
8             }
9
10            for (int i = 0; i < n - 1; i++) {
11                int minIndex = i;
12                for (int j = i + 1; j < n; j++) {
13                    if (arr[j][0] < arr[minIndex][0]) {
14                        minIndex = j;
15                    }
16                }
17                vector<int> temp = arr[i];
18                arr[i] = arr[minIndex];
19                arr[minIndex] = temp;
20            }
21
22            vector<vector<int>> ans;
23
24            for (int i = 0; i < n; i++) {
25                int ansSize = 0;
26                for (int k = 0; k < ans.size(); k++) {
27                    ansSize++;
28                }
29
30                if (ansSize == 0 || arr[i][0] > ans[ansSize - 1][1]) {
31                    ans.push_back(arr[i]);
32                }
33                else {
34                    if (arr[i][1] > ans[ansSize - 1][1]) {
35                        ans[ansSize - 1][1] = arr[i][1];
36                    }
37                }
38            }
39
40            return ans;
41        }
42    };
```

Merge two sorted arrays without extra space

Subscribe to TUF+

Hints

Company

Given two integer arrays `nums1` and `nums2`. Both arrays are sorted in **non-decreasing** order.

Merge both the arrays into a single array sorted in **non-decreasing** order.

- The final sorted array should be stored inside the array `nums1` and it should be done in-place.
- `nums1` has a length of $m + n$, where the first m elements denote the elements of `nums1` and rest are 0s.
- `nums2` has a length of n .

Example 1

Input: `nums1 = [-5, -2, 4, 5]`, `nums2 = [-3, 1, 8]`

Output: `[-5, -3, -2, 1, 4, 5, 8]`

Explanation:

The merged array is: `[-5, -3, -2, 1, 4, 5, 8]`, where `[-5, -2, 4, 5]` are from `nums1` and `[-3, 1, 8]` are from `nums2`

Example 2

Input: `nums1 = [0, 2, 7, 8]`, `nums2 = [-7, -3, -1]`

Output: `[-7, -3, -1, 0, 2, 7, 8]`

Explanation:

The merged array is: `[-7, -3, -1, 0, 2, 7, 8]`, where `[0, 2, 7, 8]` are from `nums1` and `[-7, -3, -1]` are from `nums2`

Now your turn!

Input: `nums1 = [1, 3, 5]`, `nums2 = [2, 4, 6, 7]`

Output: **Pick your answer**

[1, 2, 3, 4, 5, 6, 7]

[1, 2, 3, 4, 5, 6, 7]

Tab-1

+

```
1 class Solution {
2 public:
3     void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
4         int i = m - 1;
5         int j = n - 1;
6         int k = m + n - 1;
7         while (i >= 0 && j >= 0) {
8             if (nums1[i] > nums2[j]) {
9                 nums1[k] = nums1[i];
10                i--;
11            } else {
12                nums1[k] = nums2[j];
13                j--;
14            }
15            k--;
16        }
17        while (j >= 0) {
18            nums1[k] = nums2[j];
19            j--;
20            k--;
21        }
22    }
23 };
```

Test Cases

Case 1

Case 2

+

Reset

Accepted

NUMS1

[-5, -2, 4, 5]

Find the Duplicate Number

Subscribe to TUF+

Hints

Company

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive, there is only one repeated number in `nums` (but can be repeated more than once), return this duplicate number.

You must not modify the array (assume it is read-only), and you must use only constant extra space.

Your algorithm should run in less than $O(n^2)$ time.

Example 1

Input: `nums = [1,3,4,2,2]`

Output: 2

Example 2

Input: `nums = [3,1,3,4,2]`

Output: 3

Now your turn!

Input: `nums = [1,1]`

Output: **Pick your answer**

1

2

Constraints

- $1 \leq n \leq 10^5$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- Only one duplicate number, but it can be repeated more than once

Tab-1

+

```
1 class Solution {
2 public:
3     int findDuplicate(vector<int>& nums) {
4         // Your code goes here
5         int slow = nums[0];
6         int fast = nums[0];
7         do {
8             slow = nums[slow];
9             fast = nums[nums[fast]];
10        } while (slow != fast);
11        fast = nums[0];
12        while (slow != fast) {
13            slow = nums[slow];
14            fast = nums[fast];
15        }
16        return slow;
17    }
18 }
19 };
```

Test Cases

Case 1

Case 2

+

Reset

Accepted

NUMS

[1,3,4,2,2]

Search in a 2D matrix

Subscribe to TUF+

Hints

Company

Given a 2-D array `mat` where the elements of each row are sorted in non-decreasing order, and the first element of a row is greater than the last element of the previous row (if it exists), and an integer `target`, determine if the `target` exists in the given `mat` or not.

Example 1

Input: `mat = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12] ]`, `target = 8`

Output: True

Explanation: The `target = 8` exists in the 'mat' at index (1, 3).

Example 2

Input: `mat = [ [1, 2, 4], [6, 7, 8], [9, 10, 34] ]`, `target = 78`

Output: False

Explanation: The `target = 78` does not exist in the 'mat'. Therefore in the output, we see 'false'.

Now your turn!

Input: `mat = [ [1, 2, 4], [6, 7, 8], [9, 10, 34] ]`, `target = 7`

Output: **Pick your answer**

True

False

Constraints

- $n == \text{mat.length}$
- $m == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-10^4 \leq \text{mat}[i][j], \text{target} \leq 10^4$

Tab-1

+

```
1 class Solution{
2 public:
3     bool searchMatrix(vector<vector<int>> &mat, int target){
4         int rows = 0;
5         for (int i = 0; i < mat.size(); i++) {
6             rows++;
7         }
8         if (rows == 0) return false;
9
10        int cols = 0;
11        for (int j = 0; j < mat[0].size(); j++) {
12            cols++;
13        }
14
15        int low = 0;
16        int high = (rows * cols) - 1;
17
18        while (low <= high) {
19            int mid = low + (high - low) / 2;
20            int midElement = mat[mid / cols][mid % cols];
21
22            if (midElement == target) {
23                return true;
24            } else if (midElement < target) {
25                low = mid + 1;
26            } else {
27                high = mid - 1;
28            }
29        }
30        return false;
31    }
32 }
33 };
```

Test Cases

Pow(x,n)

Subscribe to TUF+

Hints

Company

Implement the power function $\text{pow}(x, n)$, which calculates the x raised to n i.e. x^n .

Note: In output print 4 digits places after decimal point.

Example 1

Input: $x = 2.0000$, $n = 10$
Output: 1024.0000
Explanation: Answer = $2^{10} \Rightarrow 1024$.

Example 2

Input: $x = 2.0000$, $n = -2$
Output: 0.2500
Explanation: Answer = $2^{(-2)} = 1/4 \Rightarrow 0.25$.

Now your turn!

Input: $x = 2.5000$, $n = 2$
Output: **Pick your answer**

☐ 6.5000

☐ 6.2500

☐ 7.2500

☐ 7.5000

Constraints

- $-100.0 \leq x \leq 100.0$
- $-2^{31} \leq n \leq 2^{31} - 1$
- $-10^8 \leq x^n \leq 10^8$
- Either x is not zero or $n > 0$.

Tab-1 +

```
1 class Solution {
2 public:
3     double myPow(double x, int n) {
4         //your code goes here
5         double res = 1.0;
6         long long p = n;
7
8         if (p < 0) {
9             p = -p;
10        }
11
12        while (p > 0) {
13            if (p % 2 == 1) {
14                res = res * x;
15                p = p - 1;
16            } else {
17                x = x * x;
18                p = p / 2;
19            }
20        }
21
22        if (n < 0) {
23            return (double)(1.0) / (double)(res);
24        }
25
26        return res;
27    }
28};
```

Test Cases

✓ Case 1

✓ Case 2

+

Reset

✓ Accepted

N

10

Majority Element-I

Subscribe to TUF+

Hints

Company

Given an integer array nums of size n , return the majority element of the array.

The majority element of an array is an element that appears more than $n/2$ times in the array. The array is guaranteed to have a majority element.

Example 1

Input: $\text{nums} = [7, 0, 0, 1, 7, 7, 2, 7, 7]$
Output: 7
Explanation:
The number 7 appears 5 times in the 9 sized array

Example 2

Input: $\text{nums} = [1, 1, 1, 2, 1, 2]$
Output: 1
Explanation:
The number 1 appears 4 times in the 6 sized array

Now your turn!

Input: $\text{nums} = [-1, -1, -1, -1]$
Output: **Pick your answer**

☐ 0

☐ 1

☐ -1

☐ 4

Constraints

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$

Tab-1 +

```
1 class Solution {
2 public:
3     int majorityElement(vector<int>& nums) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8
9         int count = 0;
10        int element = 0;
11        for (int i = 0; i < n; i++) {
12            if (count == 0) {
13                element = nums[i];
14                count = 1;
15            } else if (nums[i] == element) {
16                count++;
17            } else {
18                count--;
19            }
20        }
21
22        return element;
23    }
24};
```

Test Cases

✓ Case 1

✓ Case 2

+

Reset

✓ Accepted

NUMS

Tab-1

+

```
1  class Solution {
2  public:
3      vector<int> majorityElementTwo(vector<int>& nums) {
4          int n = 0;
5          for (int i = 0; i < nums.size(); i++) {
6              n++;
7          }
8
9          int cnt1 = 0;
10         int cnt2 = 0;
11         int el1 = 0;
12         int el2 = 0;
13
14         for (int i = 0; i < n; i++) {
15             if (cnt1 == 0 && nums[i] != el2) {
16                 el1 = nums[i];
17                 cnt1 = 1;
18             } else if (cnt2 == 0 && nums[i] != el1) {
19                 el2 = nums[i];
20                 cnt2 = 1;
21             } else if (nums[i] == el1) {
22                 cnt1++;
23             } else if (nums[i] == el2) {
24                 cnt2++;
25             } else {
26                 cnt1--;
27                 cnt2--;
28             }
29         }
30
31         int freq1 = 0;
32         int freq2 = 0;
33
34         for (int i = 0; i < n; i++) {
35             if (nums[i] == el1) {
36                 freq1++;
37             } else if (nums[i] == el2) {
38                 freq2++;
39             }
40         }
41
42         vector<int> result;
43
44         if (freq1 > n / 3) {
45             result.push_back(el1);
46         }
47
48         if (cnt2 > 0 && freq2 > n / 3) {
49             if (el1 != el2) {
```

```

30
31     int freq1 = 0;
32     int freq2 = 0;
33
34     for (int i = 0; i < n; i++) {
35         if (nums[i] == el1) {
36             freq1++;
37         } else if (nums[i] == el2) {
38             freq2++;
39         }
40     }
41
42     vector<int> result;
43
44     if (freq1 > n / 3) {
45         result.push_back(el1);
46     }
47
48     if (cnt2 > 0 && freq2 > n / 3) {
49         if (el1 != el2) {
50             result.push_back(el2);
51         }
52     }
53
54     return result;
55 }
56 };

```

Grid unique paths

Subscribe to TUF+

Hints

Company

Given two integers m and n , representing the number of rows and columns of a 2d array named matrix. Return the number of **unique ways** to go from the top-left cell ($matrix[0][0]$) to the bottom-right cell ($matrix[m-1][n-1]$).

Movement is allowed only in two directions from a cell: **right** and **bottom**.

Example 1

Input: $m = 3, n = 2$

Output: 3

Explanation:

There are 3 unique ways to go from the top left to the bottom right cell.

1) right -> down -> down

2) down -> right -> down

3) down -> down -> right

Example 2

Input: $m = 2, n = 4$

Output: 4

Explanation:

There are 4 unique ways to go from the top left to the bottom right cell.

1) down -> right -> right -> right

2) right -> down -> right -> right

3) right -> right -> down -> right

4) right -> right -> right -> down

Now your turn!

Input: $m = 3, n = 3$

Output: **Pick your answer**

9

6

Tab-1 +

```

1 class Solution {
2 public:
3     int uniquePaths(int m, int n) {
4         int totalSteps = m + n - 2;
5
6         int r = m - 1;
7
8         long long res = 1;
9
10        for (int i = 1; i <= r; i++) {
11            res = res * (totalSteps - r + i) / i;
12        }
13
14        return (int)res;
15    }
16 };

```

Test Cases

✓ Case 1

✓ Case 2

+

Reset

✓ Accepted

M

3

Q.Reverse Pairs

C++ Auto

```
1  class Solution {
2  public:
3      void merge(vector<int>& nums, int low, int mid, int high) {
4          vector<int> temp;
5          int left = low;
6          int right = mid + 1;
7
8          while (left <= mid && right <= high) {
9              if (nums[left] <= nums[right]) {
10                 temp.push_back(nums[left]);
11                 left++;
12             } else {
13                 temp.push_back(nums[right]);
14                 right++;
15             }
16         }
17
18         while (left <= mid) {
19             temp.push_back(nums[left]);
20             left++;
21         }
22
23         while (right <= high) {
24             temp.push_back(nums[right]);
25             right++;
26         }
27
28         for (int i = low; i <= high; i++) {
29             nums[i] = temp[i - low];
30         }
31     }
32
33     int countPairs(vector<int>& nums, int low, int mid, int high) {
34         int count = 0;
35         int right = mid + 1;
36
37         for (int i = low; i <= mid; i++) {
38             while (right <= high && (long long)nums[i] > 2 * (long long)nums[right]) {
39                 right++;
40             }
41             count = count + (right - (mid + 1));
42         }
43
44         return count;
45     }
46
47     int mergeSort(vector<int>& nums, int low, int high) {
48         int count = 0;
49         if (low >= high) {
50             return count;
51         }
52         int mid = (low + high) / 2;
53         count = mergeSort(nums, low, mid) + mergeSort(nums, mid + 1, high);
54         merge(nums, low, mid, high);
55         count = countPairs(nums, low, mid, high) + count;
56         return count;
57     }
58 }
```

```
int mergeSort(vector<int>& nums, int low, int high) {
    int count = 0;
    if (low >= high) {
        return count;
    }

    int mid = low + (high - low) / 2;

    count = count + mergeSort(nums, low, mid);
    count = count + mergeSort(nums, mid + 1, high);
    count = count + countPairs(nums, low, mid, high);

    merge(nums, low, mid, high);

    return count;
}

int reversePairs(vector<int>& nums) {
    int n = 0;
    for (int i = 0; i < nums.size(); i++) {
        n++;
    }

    return mergeSort(nums, 0, n - 1);
}

};
```

Q.Two Sum

</> Code

C++   Auto

```
1  class Solution {
2  public:
3      vector<int> twoSum(vector<int>& nums, int target) {
4          int n = 0;
5          for (int i = 0; i < nums.size(); i++) {
6              n++;
7          }
8          vector<vector<int>> pairs;
9          for (int i = 0; i < n; i++) {
10             pairs.push_back({nums[i], i});
11         }
12
13         for (int i = 0; i < n - 1; i++) {
14             int minIndex = i;
15             for (int j = i + 1; j < n; j++) {
16                 if (pairs[j][0] < pairs[minIndex][0]) {
17                     minIndex = j;
18                 }
19             }
20             vector<int> temp = pairs[i];
21             pairs[i] = pairs[minIndex];
22             pairs[minIndex] = temp;
23         }
24
25         int left = 0;
26         int right = n - 1;
27
28         while (left < right) {
29             int sum = pairs[left][0] + pairs[right][0];
30             if (sum == target) {
31                 return {pairs[left][1], pairs[right][1]};
32             } else if (sum < target) {
33                 left++;
34             } else {
35                 right--;
36             }
37         }
38
39         return {};
40     }
41 };
```

Q.Four Sum

```

        while (k < l) {
            long long sum = (long long)nums[i] + nums[j] + nums[k] + nums[l];

            if (sum == target) {
                ans.push_back({nums[i], nums[j], nums[k], nums[l]});
                k++;
                l--;
                while (k < l && nums[k] == nums[k - 1]) {
                    k++;
                }
                while (k < l && nums[l] == nums[l + 1]) {
                    l--;
                }
            } else if (sum < target) {
                k++;
            } else {
                l--;
            }
        }
    }

    return ans;
}
};

```

Description Editorial Solutions Submissions

3. Longest Substring Without Repeating Characters

Medium Topics Companies Hint

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

Input: `s = "abcabcbb"`

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: `s = "bbbbb"`

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: `s = "pwwkew"`

Output: 3

Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 \times 10^4$
- `s` consists of English letters, digits, symbols and spaces.

Code

C++ Auto

```

1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         int n = s.length();
5         for (int i = 0; i < n; i++) {
6             n++;
7         }
8         int mpp[256];
9         for (int i = 0; i < 256; i++) {
10             mpp[i] = -1;
11         }
12
13         int left = 0;
14         int right = 0;
15         int maxlen = 0;
16
17         while (right < n) {
18             if (mpp[s[right]] != -1) {
19                 if (mpp[s[right]] + 1 > left) {
20                     left = mpp[s[right]] + 1;
21                 }
22             }
23
24             mpp[s[right]] = right;
25
26             int currentLen = right - left + 1;
27             if (currentLen > maxlen) {
28                 maxlen = currentLen;
29             }
30
31             right++;
32         }
33
34         return maxlen;
35     }
36 };

```

Saved

Ln 25, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

45.1K 893 709 Online

7. Reverse Integer

Medium Topics Companies

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$
Output: 321

Example 2:

Input: $x = -123$
Output: -321

Example 3:

Input: $x = 120$
Output: 21

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

GoDaddy Get email that looks as professional as your business is.

C++ Auto

```
1 class Solution {
2 public:
3     int reverse(int x) {
4         int ans = 0;
5
6         while (x != 0) {
7             int digit = x % 10;
8
9             if (ans > 214748364 || (ans == 214748364 && digit > 7)) {
10                 return 0;
11             }
12             if (ans < -214748364 || (ans == -214748364 && digit < -8)) {
13                 return 0;
14             }
15
16             ans = ans * 10 + digit;
17             x = x / 10;
18         }
19
20         return ans;
21     }
22 };
```

Saved

Ln 8, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

$x =$

123

Output

9. Palindrome Number

Solved

Easy Topics Companies Hint

Given an integer x , return `true` if x is a *palindrome*, and `false` otherwise.

Example 1:

Input: $x = 121$
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

Follow up: Could you solve it without converting the integer to a string?

C++ Auto

```
1 class Solution {
2 public:
3     bool isPalindrome(int x) {
4         if (x < 0 || (x % 10 == 0 && x != 0)) {
5             return false;
6         }
7
8         int reversedHalf = 0;
9         while (x > reversedHalf) {
10             int digit = x % 10;
11             reversedHalf = reversedHalf * 10 + digit;
12             x = x / 10;
13         }
14         return x == reversedHalf || x == reversedHalf / 10;
15     }
16 };
```

Saved

Ln 13, Col 10

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

$x =$

121

Output

true

16K

731

306 Online

Q.Highest Occuring Element In An Array

Description Editorial Solutions Submissions

Input: nums = [1,2,4], k = 5

Output: 3

Explanation: Increment the first element three times and the second element two times to make nums = [4,4,4].
4 has a frequency of 3.

Example 2:

Input: nums = [1,4,8,13], k = 5

Output: 2

Explanation: There are multiple optimal solutions:

- Increment the first element three times to make nums = [4,4,8,13]. 4 has a frequency of 2.
- Increment the second element four times to make nums = [1,8,8,13]. 8 has a frequency of 2.
- Increment the third element five times to make nums = [1,4,13,13]. 13 has a frequency of 2.

Example 3:

Input: nums = [3,9,6], k = 2

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^9$
- $1 \leq k \leq 10^9$

GoDaddy Get email that looks as professional as your business is.

Seen this question in a real interview before? 1/6

Yes No

Accepted 276,783 / 617.2K Acceptance Rate 44.8%

Topics

5.8K 236 88 Online

Code

C++ Auto

```
1 class Solution {
2 public:
3     int maxFrequency(vector<int>& nums, int k) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8         for (int i = 0; i < n - 1; i++) {
9             int minIdx = i;
10            for (int j = i + 1; j < n; j++) {
11                if (nums[j] < nums[minIdx]) {
12                    minIdx = j;
13                }
14            }
15            int temp = nums[i];
16            nums[i] = nums[minIdx];
17            nums[minIdx] = temp;
18        }
19
20        int left = 0;
21        int maxFreq = 0;
22        long long totalSum = 0;
23
24        for (int right = 0; right < n; right++) {
25            totalSum += nums[right];
26            while ((long long)nums[right] * (right - left + 1) - totalSum >
27                k) {
28                totalSum -= nums[left];
29                left++;
30            }
31
32            int currentFreq = right - left + 1;
33            if (currentFreq > maxFreq) {
34                maxFreq = currentFreq;
35            }
36        }
37
38        return maxFreq;
39    };
}
```

Saved

Ln 25, Col 37

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

1752. Check if Array Is Sorted and Rotated

Easy Topics Companies Hint

Given an array `nums`, return `true` if the array was originally sorted in non-decreasing order, then rotated **some** number of positions (including zero). Otherwise, return `false`.

There may be **duplicates** in the original array.

Note: An array `A` rotated by `x` positions results in an array `B` of the same length such that $B[i] = A[(i+x) \% A.length]$ for every valid index `i`.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: `true`

Explanation: `[1,2,3,4,5]` is the original sorted array. You can rotate the array by `x = 2` positions to begin on the element of value 3: `[3,4,5,1,2]`.

Example 2:

Input: `nums = [2,1,3,4]`

Output: `false`

Explanation: There is no sorted array once rotated that can make `nums`.

Example 3:

Input: `nums = [1,2,3]`

Output: `true`

Explanation: `[1,2,3]` is the original sorted array. You can rotate the array by `x = 0` positions (i.e. no rotation) to make `nums`.

5.2K 431 ☆ 147 Online

Code

C++ Auto

```
1 class Solution {
2 public:
3     bool check(vector<int>& nums) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8         int count = 0;
9         for (int i = 0; i < n; i++) {
10             if (nums[i] > nums[(i + 1) % n]) {
11                 count++;
12             }
13         }
14         return count <= 1;
15     }
16 };
```

Saved

Ln 13, Col 10

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =`
`[3,4,5,1,2]`

Output

`true`

Expected

`true`

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of **unique elements** in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`

Output: `2, nums = [1,2,_]`

Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2

19.3K 1.1K ☆ 397 Online

Code

C++ Auto

```
1 class Solution {
2 public:
3     int removeDuplicates(vector<int>& nums) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8         if (n == 0) {
9             return 0;
10        }
11        int i = 0;
12        for (int j = 1; j < n; j++) {
13            if (nums[j] != nums[i]) {
14                i++;
15                nums[i] = nums[j];
16            }
17        }
18        return i + 1;
19    }
20 };
```

Saved

Ln 17, Col 10

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`nums =`
`[1,1,2]`

Output

`[1,2]`

DescriptionEditorialSolutionsSubmissions

189. Rotate Array

MediumTopicsCompaniesHint

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3`
Output: `[5,6,7,1,2,3,4]`
Explanation:
rotate 1 steps to the right: `[7,1,2,3,4,5,6]`
rotate 2 steps to the right: `[6,7,1,2,3,4,5]`
rotate 3 steps to the right: `[5,6,7,1,2,3,4]`

Example 2:

Input: `nums = [-1,-100,3,99]`, `k = 2`
Output: `[3,99,-1,-100]`
Explanation:
rotate 1 steps to the right: `[99,-1,-100,3]`
rotate 2 steps to the right: `[3,99,-1,-100]`

Constraints:

- `1 <= nums.length <= 105`
- `-231 <= nums[i] <= 231 - 1`
- `0 <= k <= 105`

Follow up:

- Try to come up with as many solutions as you can. There are at least

21.2K694226 Online

Code

C++Auto

```
1 class Solution {
2 public:
3     void reverse(vector<int>& nums, int start, int end) {
4         while (start < end) {
5             int temp = nums[start];
6             nums[start] = nums[end];
7             nums[end] = temp;
8             start++;
9             end--;
10        }
11    }
12
13    void rotate(vector<int>& nums, int k) {
14        int n = 0;
15        for (int i = 0; i < nums.size(); i++) {
16            n++;
17        }
18        k = k % n;
19        reverse(nums, 0, n - k - 1);
20        reverse(nums, n - k, n - 1);
21        reverse(nums, 0, n - 1);
22    }
23 };
```

SavedLn 20, Col 37

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

nums =

[1,2,3,4,5,6,7]

k =

3

DescriptionEditorialSolutionsSubmissions

283. Move Zeroes

EasyTopicsCompaniesHint

Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`
Output: `[1,3,12,0,0]`

Example 2:

Input: `nums = [0]`
Output: `[0]`

Constraints:

- `1 <= nums.length <= 104`
- `-231 <= nums[i] <= 231 - 1`

Follow up: Could you minimize the total number of operations done?

19.5K414212 Online

Code

C++Auto

```
1 class Solution {
2 public:
3     void moveZeroes(vector<int>& nums) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8         int j = -1;
9         for (int i = 0; i < n; i++) {
10            if (nums[i] == 0) {
11                j = i;
12                break;
13            }
14        }
15        if (j == -1) {
16            return;
17        }
18        for (int i = j + 1; i < n; i++) {
19            if (nums[i] != 0) {
20                int temp = nums[i];
21                nums[i] = nums[j];
22                nums[j] = temp;
23                j++;
24            }
25        }
26    }
27 };
```

SavedLn 17, Col 10

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

nums =

[0,1,0,3,12]

Government of India

Census 2027 (May)

Open

DescriptionEditorialSolutionsSubmissions

485. Max Consecutive Ones

EasyTopicsCompaniesHint

Given a binary array `nums`, return the maximum number of consecutive `1`'s in the array.

Example 1:

Input: `nums = [1,1,0,1,1,1]`
Output: `3`
Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.

Example 2:

Input: `nums = [1,0,1,1,0,1]`
Output: `2`

Constraints:

- `1 <= nums.length <= 105`
- `nums[i]` is either `0` or `1`.

 Government of India
Census 2027 (May)

Open

Seen this question in a real interview before? 1/6

YesNo

Accepted 2,318,788 / 3.6M | Acceptance Rate 65.2%

6.9K142584 Online

Code

C++Auto

```
1 class Solution {
2 public:
3     int findMaxConsecutiveOnes(vector<int>& nums) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8
9         int maxCount = 0;
10        int currentCount = 0;
11
12        for (int i = 0; i < n; i++) {
13            if (nums[i] == 1) {
14                currentCount++;
15                if (currentCount > maxCount) {
16                    maxCount = currentCount;
17                }
18            } else {
19                currentCount = 0;
20            }
21        }
22
23        return maxCount;
24    }
25 };
```

SavedLn 23, Col 25

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

nums =
[1,1,0,1,1,1]

Output

DescriptionAcceptedEditorialSolutionsSubm

136. Single Number

EasyTopicsCompaniesHint

Given a **non-empty** array of integers `nums`, every element appears *twice* except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`
Output: `1`

Example 2:

Input: `nums = [4,1,2,1,2]`
Output: `4`

Example 3:

Input: `nums = [1]`
Output: `1`

Constraints:

- `1 <= nums.length <= 3 * 104`
- `-3 * 104 <= nums[i] <= 3 * 104`
- Each element in the array appears twice except for one element which appears only once.

18.9K324137 Online

Code

C++Auto

```
1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         int xorResult = 0;
5         for (int num : nums) {
6             xorResult ^= num;
7         }
8         return xorResult;
9     }
10 };
```

SavedLn 10, Col 3

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

nums =
[2,2,1]

Output

2149. Rearrange Array Elements by Sign

Solved 

Medium Topics Companies Hint

You are given a **0-indexed** integer array `nums` of **even** length consisting of an **equal** number of positive and negative integers.

You should return the array of `nums` such that the array follows the given conditions:

1. Every **consecutive pair** of integers have **opposite signs**.
2. For all integers with the same sign, the **order** in which they were present in `nums` is **preserved**.
3. The rearranged array begins with a positive integer.

Return the modified array after rearranging the elements to satisfy the aforementioned conditions.

Example 1:

Input: `nums = [3,1,-2,-5,2,-4]`

Output: `[3,-2,1,-5,2,-4]`

Explanation:

The positive integers in `nums` are `[3,1,2]`. The negative integers are `[-2,-5,-4]`.

The only possible way to rearrange them such that they satisfy all conditions is `[3,-2,1,-5,2,-4]`.

Other ways such as `[1,-2,2,-5,3,-4]`, `[3,1,2,-2,-5,-4]`, `[-2,3,-5,1,-4,2]` are incorrect because they do not satisfy one or more conditions.

Example 2:

Input: `nums = [-1,1]`

Output: `[1,-1]`

4.2K 261 51 Online

C++ Auto

```
1 class Solution {
2 public:
3     vector<int> rearrangeArray(vector<int>& nums) {
4         int n = nums.size();
5         vector<int> ans(n);
6         int posIndex = 0, negIndex = 1;
7
8         for (int i = 0; i < n; i++) {
9             if (nums[i] > 0) {
10                 ans[posIndex] = nums[i];
11                 posIndex += 2;
12             } else {
13                 ans[negIndex] = nums[i];
14                 negIndex += 2;
15             }
16         }
17         return ans;
18     }
19 };
```

Saved

Ln 19, Col 3

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[3,1,-2,-5,2,-4]

Output

Q.three Sum

C++ Auto

```
1  class Solution {
2  public:
3      vector<vector<int>> threeSum(vector<int>& nums) {
4          int n = 0;
5          for (int i = 0; i < nums.size(); i++) {
6              n++;
7          }
8          for (int i = 0; i < n - 1; i++) {
9              int minIdx = i;
10             for (int j = i + 1; j < n; j++) {
11                 if (nums[j] < nums[minIdx]) {
12                     minIdx = j;
13                 }
14             }
15             int temp = nums[i];
16             nums[i] = nums[minIdx];
17             nums[minIdx] = temp;
18         }
19
20         vector<vector<int>> ans;
21         for (int i = 0; i < n; i++) {
22             if (i > 0 && nums[i] == nums[i - 1]) {
23                 continue;
24             }
25
26             int j = i + 1;
27             int k = n - 1;
28
29             while (j < k) {
30                 int sum = nums[i] + nums[j] + nums[k];
31
32                 if (sum == 0) {
33                     ans.push_back({nums[i], nums[j], nums[k]});
34                     j++;
35                     k--;
36                     while (j < k && nums[j] == nums[j - 1]) {
37                         j++;
38                     }
39                     while (j < k && nums[k] == nums[k + 1]) {
40                         k--;
41                     }
42                 } else if (sum < 0) {
```

```

33         ans.push_back({nums[i], nums[j], nums[k]});
34         j++;
35         k--;
36         while (j < k && nums[j] == nums[j - 1]) {
37             j++;
38         }
39         while (j < k && nums[k] == nums[k + 1]) {
40             k--;
41         }
42     } else if (sum < 0) {
43         j++;
44     } else {
45         k--;
46     }
47 }
48 }
49
50 return ans;
51 }
52 };

```

Description
Editorial
Solutions
Submissions

152. Maximum Product Subarray

Medium
Topics
Companies

Given an integer array `nums`, find a **subarray** that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: `nums = [2,3,-2,4]`
Output: 6
Explanation: [2,3] has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 \times 10^4$
- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of `nums` is **guaranteed** to fit in a **32-bit** integer.

C++ Auto

```

1 class Solution {
2 public:
3     int maxProduct(vector<int>& nums) {
4         int n = 0;
5         for (int i = 0; i < nums.size(); i++) {
6             n++;
7         }
8         long long maxAns = nums[0];
9         long long prefix = 1;
10        long long suffix = 1;
11
12        for (int i = 0; i < n; i++) {
13            if (prefix == 0) prefix = 1;
14            if (suffix == 0) suffix = 1;
15            prefix = prefix * nums[i];
16            suffix = suffix * nums[n - i - 1];
17            if (prefix > maxAns) maxAns = prefix;
18            if (suffix > maxAns) maxAns = suffix;
19            if (prefix > 2147483647 || prefix < -2147483648) prefix = nums[i];
20            if (suffix > 2147483647 || suffix < -2147483648) suffix = nums[n - i - 1];
21        }
22        return (int)maxAns;
23    }
24 };
25

```

Saved Ln 18, Col 50

Testcase
Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[2,3,-2,4]